

```
In [8]: # SpaceX Launch Success Prediction
# Data Science Methodology Project Template (Corrected Version)

# -----
# 1. Import Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# -----
# 2. Load Dataset
url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DSE103/Module%204/data/SpaceX_launch_success.csv'
df = pd.read_csv(url)

# Quick look at the data
print(df.head())
print(df.columns)

# -----
# 3. Data Wrangling
# Check for missing values
print(df.isnull().sum())

# Drop missing values if any
df = df.dropna()

# Optional: Feature Engineering
# Create 'PayloadCategory' based on payload mass
df['PayloadCategory'] = pd.cut(df['PayloadMass'],
                               bins=[0, 2000, 4000, 6000, 10000],
                               labels=['Light', 'Medium', 'Heavy', 'Very Heavy'])

# -----
# 4. Exploratory Data Analysis (EDA)

# Scatter plot: Payload Mass vs Launch Success
plt.figure(figsize=(8,6))
sns.scatterplot(data=df, x='PayloadMass', y='Class', hue='BoosterVersion')
plt.title('Payload Mass vs Launch Success')
plt.xlabel('Payload Mass (kg)')
plt.ylabel('Launch Success (1 = Success, 0 = Failure)')
plt.show()

# Bar plot: Launch Site vs Success Rate
site_success = df.groupby('LaunchSite')['Class'].mean().reset_index()
plt.figure(figsize=(8,6))
sns.barplot(data=site_success, x='LaunchSite', y='Class')
plt.title('Launch Site vs Average Success Rate')
plt.ylabel('Average Success Rate')
```

```
plt.xticks(rotation=45)
plt.show()

# Heatmap: Correlation Matrix
corr_matrix = df[['PayloadMass', 'Class']].corr()
plt.figure(figsize=(6,4))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()

# -----
# 5. Feature Selection and Model Development

# Select features and target
features = ['PayloadMass', 'LaunchSite', 'BoosterVersion']
X = pd.get_dummies(df[features])
y = df['Class']

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# Model Training
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# -----
# 6. Model Evaluation

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# Confusion Matrix
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# -----
# 7. Feature Importance

# Analyze feature importance
importances = model.feature_importances_
feature_names = X.columns
feature_importance_df = pd.DataFrame({'feature': feature_names, 'importance': importances})
feature_importance_df = feature_importance_df.sort_values('importance', ascending=False)

# Plot Feature Importances
plt.figure(figsize=(10,6))
sns.barplot(data=feature_importance_df.head(10), x='importance', y='feature')
plt.title('Top 10 Feature Importances')
```

```
plt.show()

# -----
# 8. Model Tuning with GridSearchCV (Optional)

# Hyperparameter tuning
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(estimator=RandomForestClassifier(random_state=42),
                           param_grid=param_grid,
                           cv=5, n_jobs=-1, verbose=2)
grid_search.fit(X_train, y_train)

print("\nBest Parameters from GridSearchCV:\n", grid_search.best_params_)

# Retrain with best parameters
best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(X_test)

# Evaluation of best model
print("\nClassification Report for Best Model:\n")
print(classification_report(y_test, y_pred_best))

# -----
# 9. Conclusion
print("\nProject Completed: SpaceX Launch Success Prediction Done!")
print("Key insights: PayloadMass, LaunchSite, and BoosterVersion strongly impact")
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	\
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	

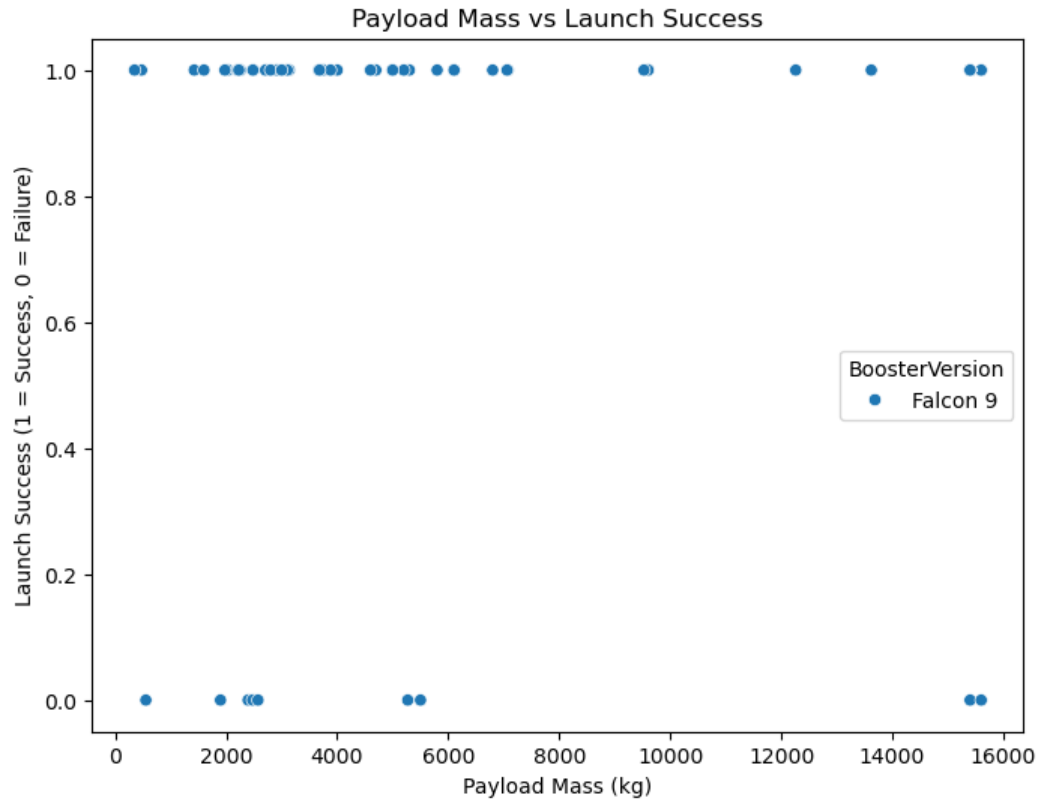
	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	\
0	None	None	1	False	False	NaN	1.0	
1	None	None	1	False	False	NaN	1.0	
2	None	None	1	False	False	NaN	1.0	
3	False	Ocean	1	False	False	NaN	1.0	
4	None	None	1	False	False	NaN	1.0	

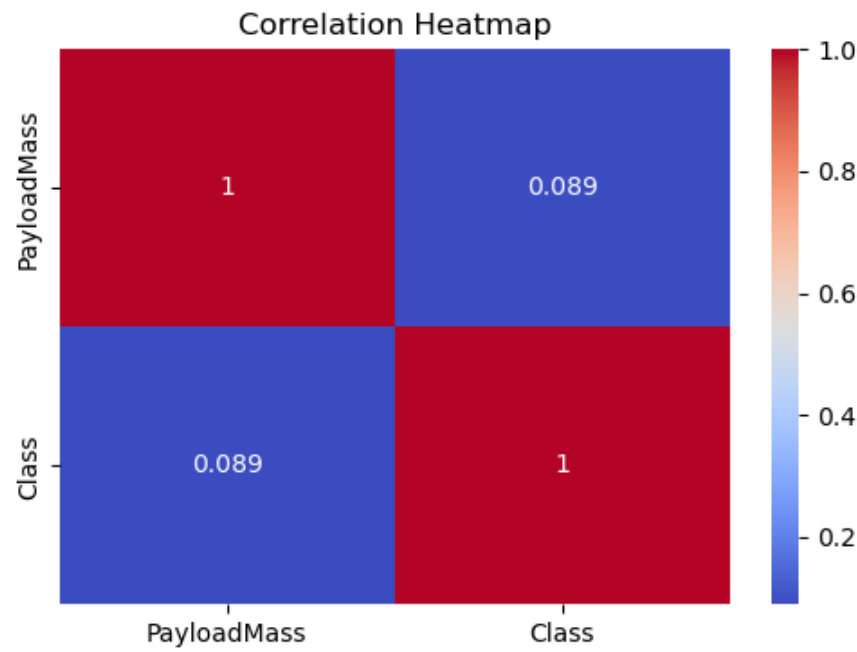
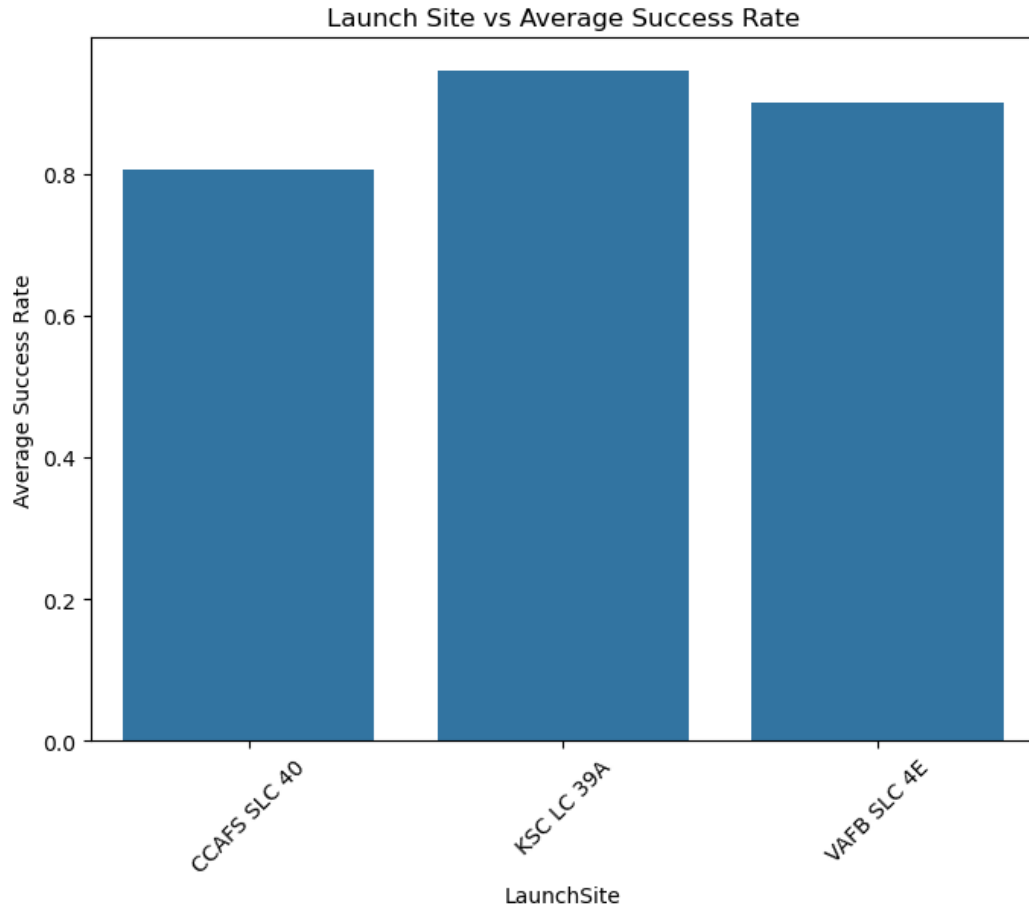
	ReusedCount	Serial	Longitude	Latitude	Class
0	0	B0003	-80.577366	28.561857	0
1	0	B0005	-80.577366	28.561857	0
2	0	B0007	-80.577366	28.561857	0
3	0	B1003	-120.610829	34.632093	0
4	0	B1004	-80.577366	28.561857	0

```

Index(['FlightNumber', 'Date', 'BoosterVersion', 'PayloadMass', 'Orbit',
      'LaunchSite', 'Outcome', 'Flights', 'GridFins', 'Reused', 'Legs',
      'LandingPad', 'Block', 'ReusedCount', 'Serial', 'Longitude', 'Latitude',
      'Class'],
      dtype='object')
FlightNumber      0
Date              0
BoosterVersion    0
PayloadMass       0
Orbit             0
LaunchSite        0
Outcome           0
Flights           0
GridFins          0
Reused            0
Legs              0
LandingPad        26
Block             0
ReusedCount       0
Serial            0
Longitude         0
Latitude          0
Class            0
dtype: int64

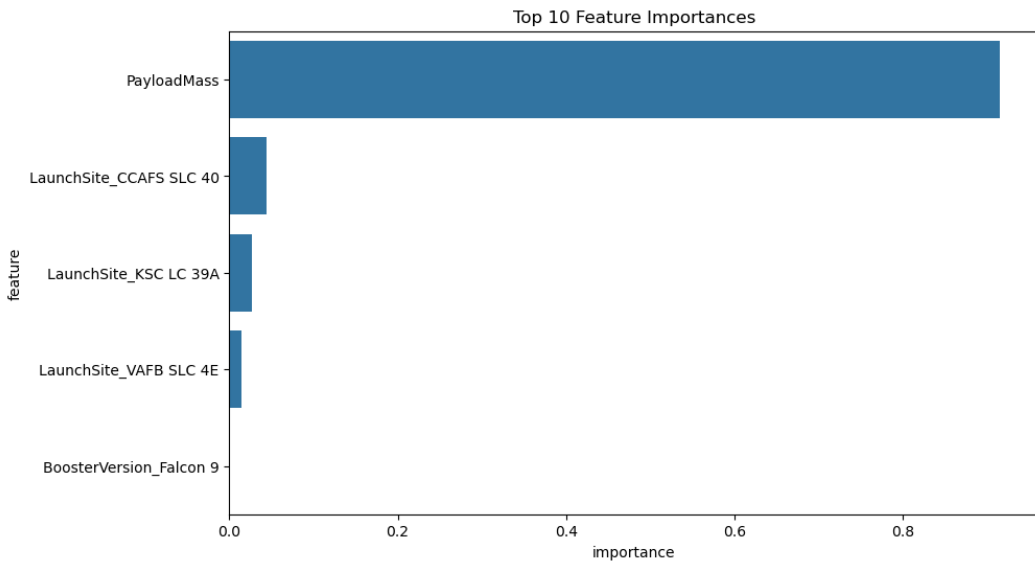
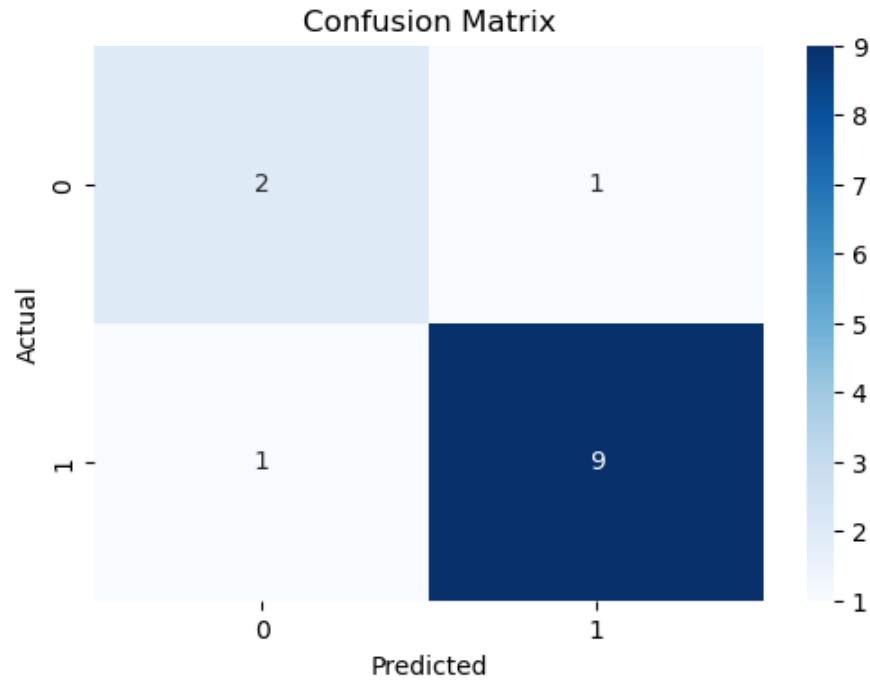
```





Classification Report:

	precision	recall	f1-score	support
0	0.67	0.67	0.67	3
1	0.90	0.90	0.90	10
accuracy			0.85	13
macro avg	0.78	0.78	0.78	13
weighted avg	0.85	0.85	0.85	13



Fitting 5 folds for each of 36 candidates, totalling 180 fits

Best Parameters from GridSearchCV:

```
{'max_depth': None, 'min_samples_split': 10, 'n_estimators': 100}
```

Classification Report for Best Model:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	3
1	0.77	1.00	0.87	10
accuracy			0.77	13
macro avg	0.38	0.50	0.43	13
weighted avg	0.59	0.77	0.67	13

Project Completed: SpaceX Launch Success Prediction Done!

Key insights: PayloadMass, LaunchSite, and BoosterVersion strongly impact launch success.

C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
In [9]: # SpaceX Launch Success Prediction
# Data Science Methodology Project Template (Corrected Version)

# -----
# 1. Import Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import joblib

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# -----
# 2. Load Dataset
url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-D
df = pd.read_csv(url)
```



```
# Quick Look at the data
display(df.head())
print(df.columns)

# -----
# 3. Data Wrangling
# Check for missing values
print(df.isnull().sum())

# Drop missing values if any
df = df.dropna()

# Optional: Feature Engineering
df['PayloadCategory'] = pd.cut(
    df['PayloadMass'], bins=[0, 2000, 4000, 6000, 10000],
    labels=['Light', 'Medium', 'Heavy', 'Very Heavy']
)

# -----
# 4. Exploratory Data Analysis (EDA)

# Scatter plot: Payload Mass vs Launch Success
plt.figure(figsize=(8,6))
sns.scatterplot(data=df, x='PayloadMass', y='Class', hue='BoosterVersion')
plt.title('Payload Mass vs Launch Success')
plt.xlabel('Payload Mass (kg)')
plt.ylabel('Launch Success (1 = Success, 0 = Failure)')
plt.show()

# Bar plot: Launch Site vs Success Rate
site_success = df.groupby('LaunchSite')['Class'].mean().reset_index()
plt.figure(figsize=(8,6))
sns.barplot(data=site_success, x='LaunchSite', y='Class')
plt.title('Launch Site vs Average Success Rate')
plt.ylabel('Average Success Rate')
plt.xticks(rotation=45)
plt.show()

# Heatmap: Correlation Matrix
corr_matrix = df[['PayloadMass', 'Class']].corr()
plt.figure(figsize=(6,4))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()

# -----
# 5. Feature Selection and Model Development

features = ['PayloadMass', 'LaunchSite', 'BoosterVersion']
X = pd.get_dummies(df[features])
y = df['Class']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# -----
# 6. Model Evaluation

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# -----
# 7. Feature Importance

importances = model.feature_importances_
feature_names = X.columns
feature_importance_df = pd.DataFrame({'feature': feature_names, 'importance': importances})
feature_importance_df = feature_importance_df.sort_values('importance', ascending=False)

plt.figure(figsize=(10,6))
sns.barplot(data=feature_importance_df.head(10), x='importance', y='feature')
plt.title('Top 10 Feature Importances')
plt.show()

# -----
# 8. Model Tuning with GridSearchCV (Optional)

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5, n_jobs=-1, verbose=2
)
grid_search.fit(X_train, y_train)

print("\nBest Parameters from GridSearchCV:\n", grid_search.best_params_)

best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(X_test)

print("\nClassification Report for Best Model:\n")
print(classification_report(y_test, y_pred_best))

# -----
# 9. Conclusion
```

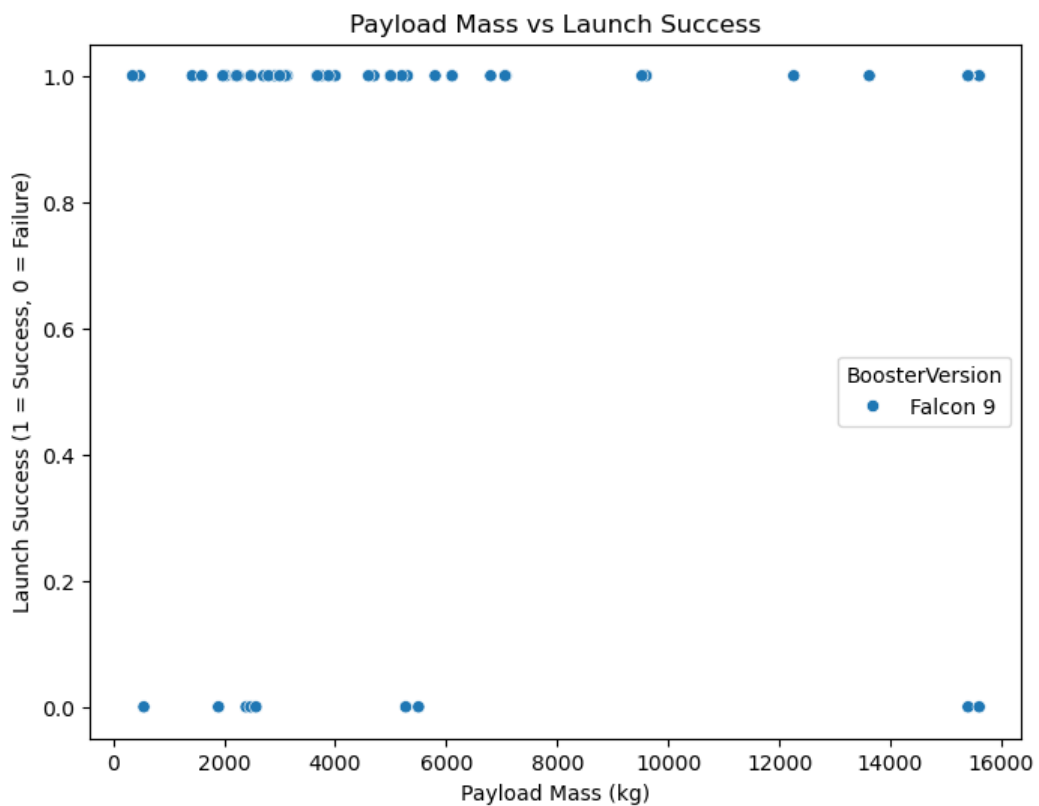
```
print("\nProject Completed: SpaceX Launch Success Prediction Done!")
print("Key insights: PayloadMass, LaunchSite, and BoosterVersion strongly impact")

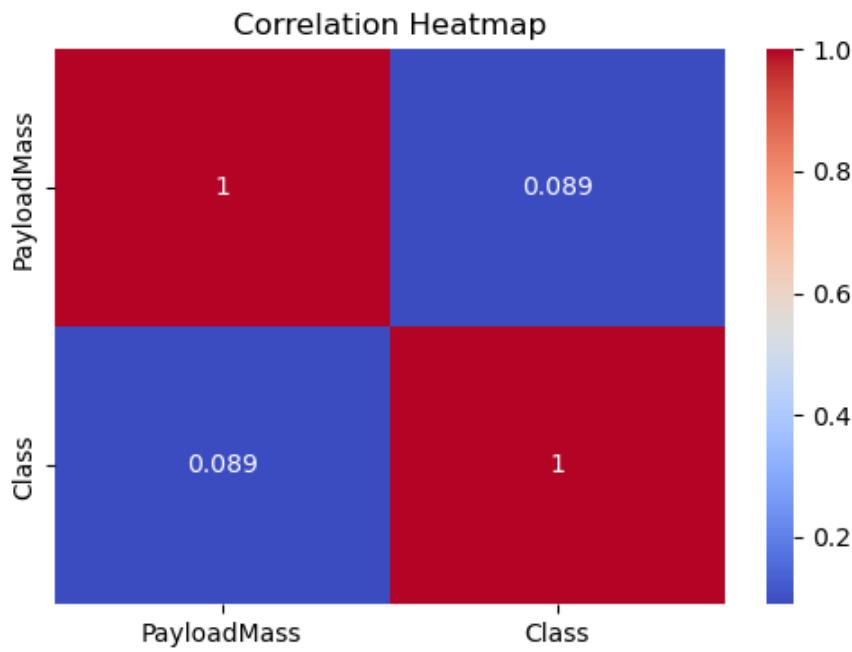
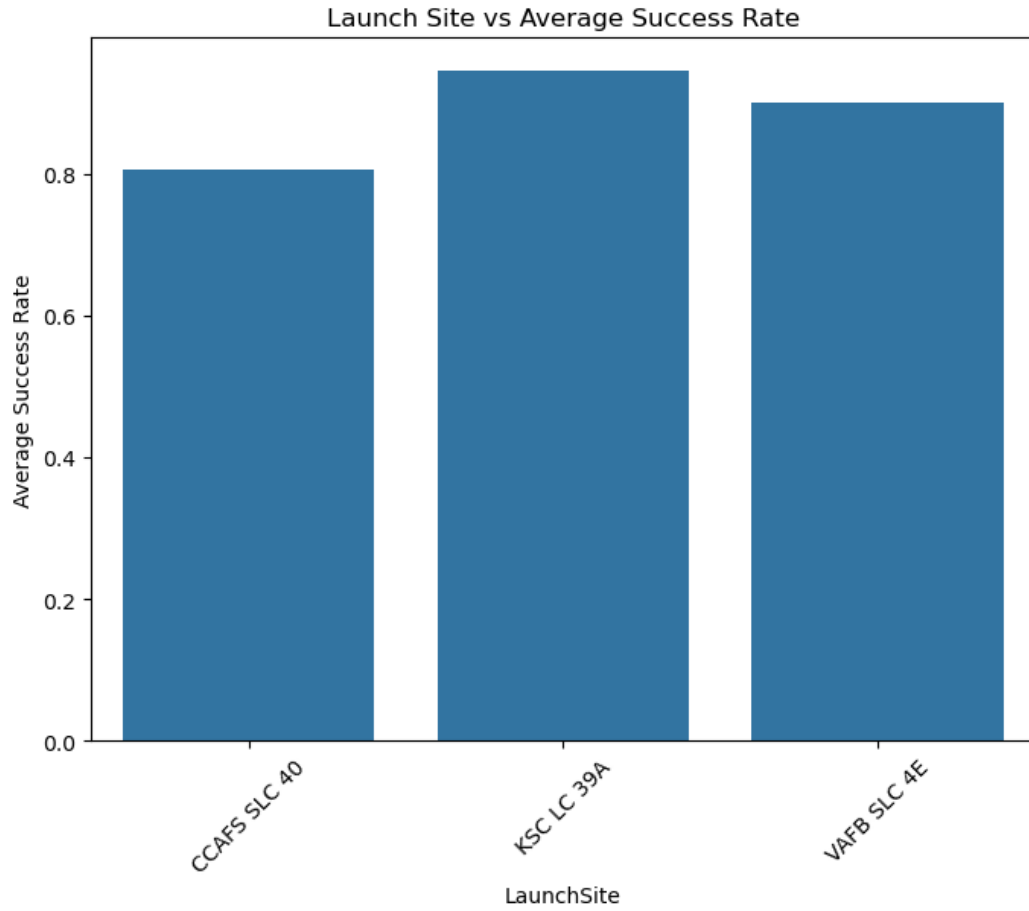
# -----
# 10. Export the Tuned Model
# Save the best model to a file for later use
joblib.dump(best_model, 'best_spaceX_launch_model.joblib')
print("Model exported to 'best_spaceX_launch_model.joblib'")

# To Load the model Later:
# loaded_model = joblib.load('best_spaceX_launch_model.joblib')
# sample_preds = loaded_model.predict(X_test.iloc[:5])
# print("Sample predictions:", sample_preds)
```

FlightNumber		Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None
	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None
	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None
	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean
	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None

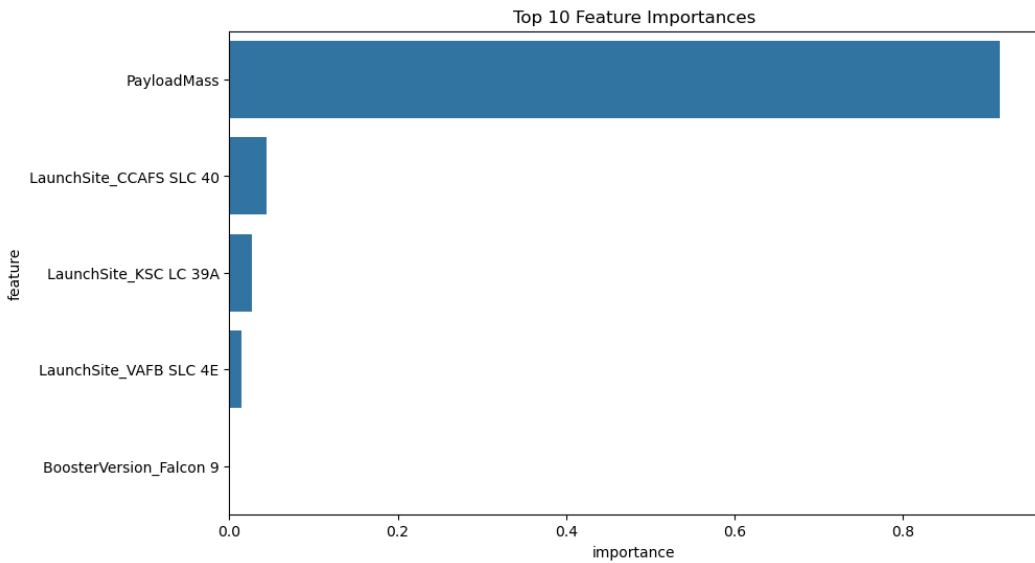
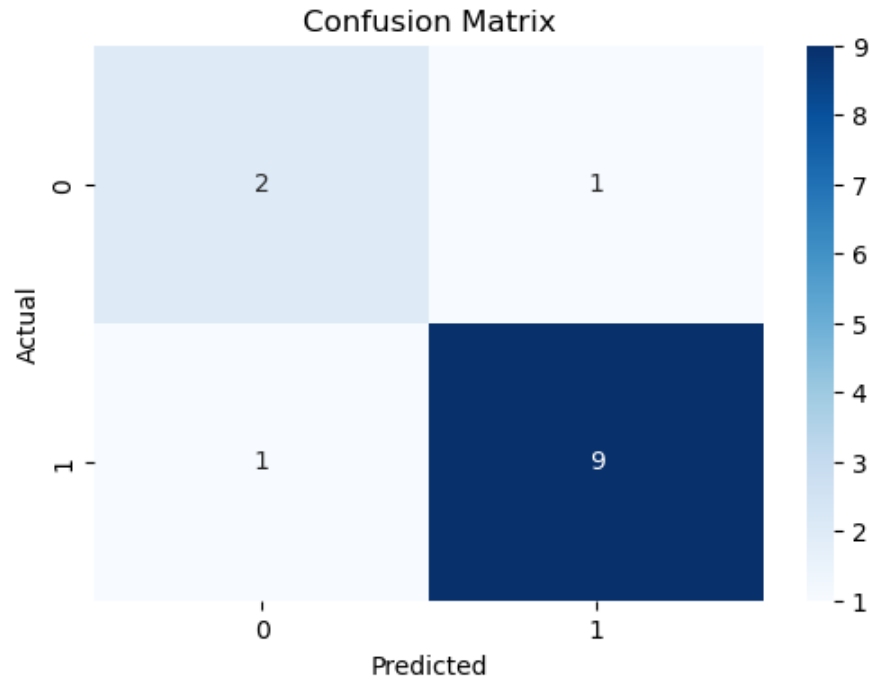
```
Index(['FlightNumber', 'Date', 'BoosterVersion', 'PayloadMass', 'Orbit',
      'LaunchSite', 'Outcome', 'Flights', 'GridFins', 'Reused', 'Legs',
      'LandingPad', 'Block', 'ReusedCount', 'Serial', 'Longitude', 'Latitude',
      'Class'],
      dtype='object')
FlightNumber      0
Date              0
BoosterVersion    0
PayloadMass       0
Orbit             0
LaunchSite        0
Outcome           0
Flights           0
GridFins          0
Reused            0
Legs              0
LandingPad        26
Block             0
ReusedCount       0
Serial            0
Longitude         0
Latitude          0
Class             0
dtype: int64
```





Classification Report:

	precision	recall	f1-score	support
0	0.67	0.67	0.67	3
1	0.90	0.90	0.90	10
accuracy			0.85	13
macro avg	0.78	0.78	0.78	13
weighted avg	0.85	0.85	0.85	13



Fitting 5 folds for each of 36 candidates, totalling 180 fits

Best Parameters from GridSearchCV:
{'max_depth': None, 'min_samples_split': 10, 'n_estimators': 100}

Classification Report for Best Model:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	3
1	0.77	1.00	0.87	10
accuracy			0.77	13
macro avg	0.38	0.50	0.43	13
weighted avg	0.59	0.77	0.67	13

Project Completed: SpaceX Launch Success Prediction Done!
Key insights: PayloadMass, LaunchSite, and BoosterVersion strongly impact launch success.

C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

 _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

 _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

 _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
Model exported to 'best_spaceX_launch_model.joblib'

In []: